

Nom :	Prénom :
-------	----------

BACCALAUREAT GENERAL

EPREUVE D'ENSEIGNEMENT DE SPECIALITE

SESSION 2024

NUMERIQUE ET SCIENCES INFORMATIQUES

Jeudi 28 Mars 2024

Durée de l'épreuve 3 heures 30

L'usage de la calculatrice n'est pas autorisé

Dès que ce sujet vous est remis, assurez-vous qu'il est complet.

Ce sujet comporte 16 pages numérotées de 1/16 à 16/16

Le candidat traitera l'intégralité du sujet en répondant sur ce document.

Si la place qui vous a été laissée ne suffit pas pour répondre, vous pouvez utiliser les deux dernières pages de ce document ou utiliser une copie double que vous joindrez à ce document.

On considère un réseau local **N1** constitué de trois ordinateurs **M1**, **M2** et **M3** dont les adresses IP sont les suivantes :

- *M1*: 192.168.1.1/24
- *M2*: 192.168.1.2/24
- *M3*: 192.168.2.3 /24

PARTIE A

1. Donner en notation pointée l'adresse du masque de sous-réseau

2. Donner en notation décimale l'adresse du réseau dans lequel est la machine 1

3. Les trois machines sont-elles toutes sur le même réseau ? Justifier

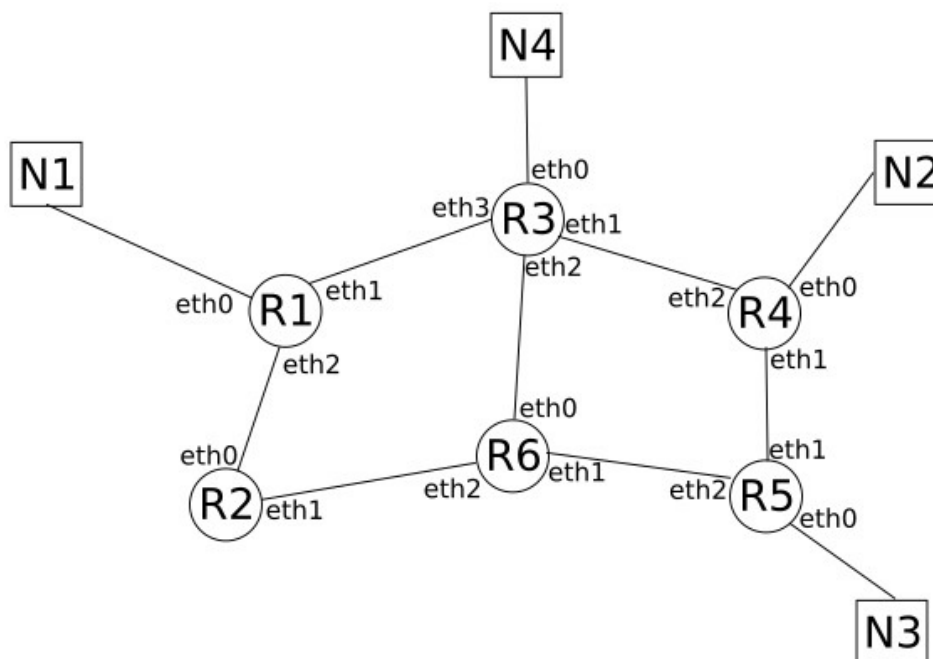
Dans la suite de l'exercice, on ajoute un routeur **R1** au réseau **N1**.

4. Pour quelle raison est-il nécessaire d'avoir au minimum deux interfaces réseaux dans un routeur ?

5. Donner une adresse IP possible pour le routeur **R1**

PARTIE B

Le réseau **N1** est maintenant relié à d'autres réseaux locaux (**N2**, **N3** et **N4**) par l'intermédiaire d'une série de routeurs (**R1**, **R2**, etc.)



On utilise dans un premier temps le protocole de routage **RIP** (Routing Information Protocol). On rappelle que dans ce protocole la métrique de la table de routage correspond au nombre de routeurs à traverser pour atteindre sa destination. La table de routage du routeur **R1** est donné dans le tableau suivant :

Table de routage de R1		
Destination	Interface de sortie	Métrique
N1	Eth0	0
N2	Eth1	2
N3	Eth1	3
N4	Eth1	1

6. Déterminer le chemin parcouru par un paquet de données pour aller d'une machine du réseau **N1** à une machine du réseau **N2**

Le routeur **R3** tombe en panne. Après quelques minutes, la table de routage de **R1** est modifiée.

7. Dresser la table de routage du routeur **R1** suite à la panne du routeur **R3**

Table de routage de R1		
Destination	Interface de sortie	Métrique
N1		
N2		
N3		

PARTIE C

Le routeur **R3** est de nouveau fonctionnel. Dans la suite de l'exercice on utilise le protocole de routage OSPF (Open Shortest Path First). On rappelle que dans ce protocole, la métrique de la table de routage correspond à la somme des coûts

$$cout = \frac{10^8}{d}$$

où d est la bande passante d'une liaison en bit/s

On donne les valeurs suivantes :

- Fibre : 1Gbit/s
- Fast Ethernet : 100 Mbit/s
- Ethernet : 10 Mbit/s

8. Calculer le coût de chacune de ces liaisons de communication

Liaison	Coût
Fibre	
Fast-Ethernet	
Ethernet	

On donne la table de routage du routeur R1 :

Destination	Interface de sortie	Métrique
N1	Eth0	0
N2	Eth1	10,1
N2	Eth2	1,3
N3	Eth1	11,1
N3	Eth2	0,3
N4	Eth1	10
N4	Eth2	1,2

9. Déterminer les types de connexion entre les différents routeurs : (mettre une croix)

LIAISON		Fibre	Fast-Ethernet	Ethernet
R1	R2			
R1	R3			
R2	R6			
R3	R4			
R3	R6			
R4	R5			
R5	R6			

On pourra utiliser dans cet exercice les mots du langage SQL suivant :

SELECT, FROM, WHERE, JOIN, INSERT INTO, VALUES, COUNT, UPDATE, SET

Afin de pouvoir gérer un site de covoiturage, on utilise une base de données contenant les relations LIEU et COMMUNE. Le schéma relationnel, où les clés primaires sont soulignées et les clés étrangères sont précédées du symbole #, est le suivant :

- LIEU(id_lieu : entier, ad_lieu : texte, #code_insee : texte, nb_places : entier, type : texte)
- COMMUNE (code_insee : texte, nom : texte, departement : texte, region : texte, latitude : décimal, longitude : décimal)

On donne un extrait des deux premières lignes de ces relations :

LIEU

id_lieu	ad_lieu	code_insee	nb_places	type
1	"Place De La Fontaine"	"01001"	5	"Supermarché"
2	"La Boisse"	"01049"	100	"Parking municipal"

COMMUNE

code_insee	nom	departement	Region	latitude	longitude
"01001"	"L'ABERGEMENT-CLEMENCIAT"	"AIN"	"RHONE-ALPES"	46.1534	4.9261
"01002"	"L'ABERGEMENT-DE-VAREY"	"AIN"	"RHONE-ALPES"	46.0092	5.4280

10. On se place dans la relation COMMUNE. Expliquer pourquoi deux communes ne peuvent pas posséder le même code INSEE code_insee

11. Écrire une requête SQL permettant d'obtenir l'identifiant `id_lieu` et le code INSEE `code_insee` des lieux de covoiturage de type « Sortie autoroute »

12. Écrire une requête SQL permettant de donner les adresses des lieux se trouvant dans le département « JURA »

La commune INSEE « 40714 » vient d'être intégrée à la commune voisine « 40146 ». En conséquence, il faut mettre à jour la base de données

13. La requête suivante a été saisie. Une erreur a été renvoyée par le logiciel. Expliquer pourquoi

```
DELETE FROM COMMUNE  
WHERE code_insee = "40714" ;
```

14. Les informations liées à la commune de code INSEE « 40714 » devront désormais être rattachées à la commune de code INSEE « 40146 ». Écrire une requête permettant de mettre à jour la table LIEU

On cherche à obtenir un mélange d'une liste comportant un nombre pair d'éléments.

Dans cet exercice, on notera N le nombre d'éléments de la liste à mélanger.

La méthode de mélange utilisée dans cette partie est inspirée d'un mélange de jeux de cartes :

- On sépare la liste en deux piles :
- à gauche, la première pile contient les $N/2$ premiers éléments de la liste ;
- à droite, la deuxième pile contient les $N/2$ derniers éléments de la liste.
- On crée une liste vide.
- On prend alors le sommet de la pile de gauche et on le met en début de liste.
- On prend ensuite le sommet de la pile de droite que l'on ajoute à la liste et ainsi de suite jusqu'à ce que les piles soient vides.

Par exemple, si on applique cette méthode de mélange à la liste ['V', 'D', 'R', '3', '7', '10'], on obtient pour le partage de la liste en 2 piles :

Pile gauche	Pile droite
'R'	'10'
'D'	'7'
'V'	'3'

La nouvelle liste à la fin du mélange sera donc ['R', '10', 'D', '7', 'V', '3'].

15. Que devient la liste ['7', '8', '9', '10', 'V', 'D', 'R', 'A'] si on lui applique cette méthode de mélange ?

On considère que l'on dispose de la structure de données de type pile, munie des seules instructions suivantes :

- `p = Pile()` : crée une pile vide nommée `p`
- `p.est_vide()` : renvoie Vrai si la liste est vide, Faux sinon
- `p.empiler(e)` : ajoute l'élément `e` dans la pile
- `e = p.depiler()` : retire le dernier élément ajouté dans la pile et le retourne (et l'affecte à la variable `e`)
- `p2 = p.copier()` : renvoie une copie de la pile `p` sans modifier la pile `p` et l'affecte à une nouvelle pile `p2`

16. Compléter une fonction `liste_vers_pile(L)` qui prend en paramètre une liste et renvoie une pile. L'indice 0 de la liste devra se trouver en bas de la pile

```
def liste_vers_pile(L):  
    ''' Prend en paramètre une liste - Renvoie une pile  
        Le premier élément de la liste est en bas de la pile'''
```

On considère la fonction suivante qui partage une liste en deux piles. Lors de sa mise au point et pour aider au débogage, des appels à la fonction `affichage_pile` ont été insérés. La fonction `affichage_pile(p)` affiche la pile `p` à l'écran verticalement sous la forme suivante :

dernier élément empilé
...
...
premier élément empilé

```
def partage(L):  
    N = len(L)  
    p_gauche = Pile()  
    p_droite = Pile()  
    for i in range(N//2):  
        p_gauche.empile(L[i])  
    for i in range(N//2, N):  
        p_droite.empile(L[i])  
    affichage_pile(p_gauche)  
    affichage_pile(p_droite)  
    return p_gauche, p_droite
```

17. Quels affichages obtient-on à l'écran lors de l'exécution de l'instruction :
`partage([1,2,3,4,5,6])` ?

18. On considère maintenant deux piles p1 et p2 de même taille. On cherche à fusionner ces deux piles en alternant un à un les éléments de la pile p1 et de la pile p2.

Écrire une fonction `fusion(p1, p2)` qui renvoie une liste construite à partir de ces deux piles p1 et p2.

```
def fusion(p1, p2):  
    ''' Fusionne la pile p1 et la pile p2  
    Alterne une valeur de p1 et une de p2  
    Renvoie une liste '''
```

EXERCICE 4

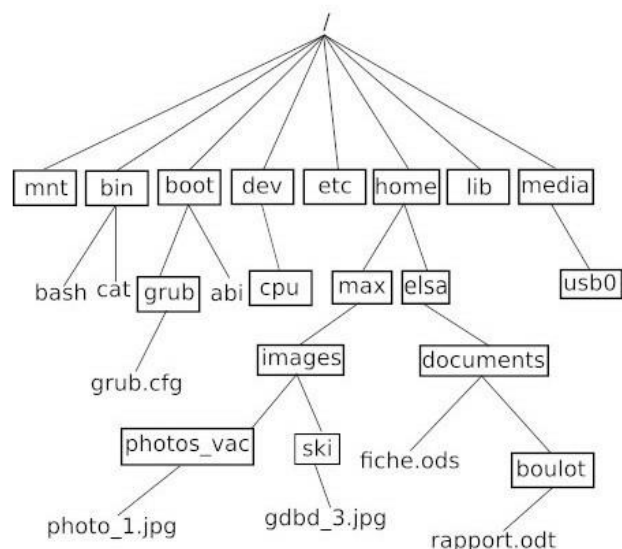
4 points

Cet exercice porte sur les systèmes d'exploitation, les structures de données (type LIFO et FIFO) et les processus

PARTIE A

On donne ci-contre une arborescence de fichiers sur un système GNU/Linux (les noms encadrés représentent des répertoires, les noms non encadrés représentent des fichiers,

/ correspond à la racine du système de fichiers) :



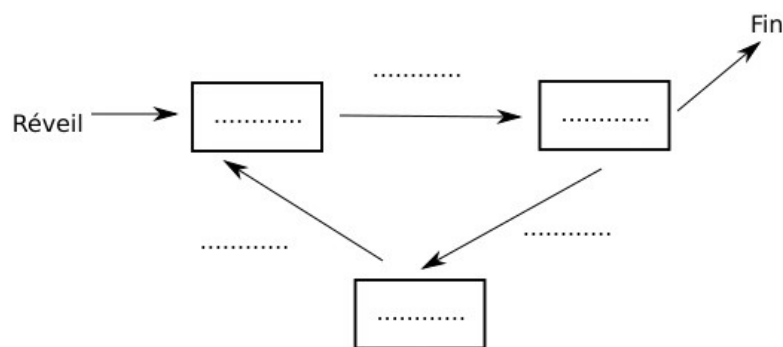
19. Donner le chemin absolu du fichier nommé gdbd_3.jpg

20. On suppose que le répertoire courant est grub. Indiquer le chemin relatif du fichier fiche.ods

Dans la suite de l'exercice, on s'intéresse aux processus. On considère qu'un monoprocesseur est utilisé. On rappelle qu'un processus est un programme en cours d'exécution. Un processus est soit élu, soit bloqué, soit prêt.

21. Compléter le schéma ci-dessous avec les termes suivants :

Élu, bloqué, prêt, élection, blocage, déblocage



PARTIE B

L'ordonnanceur peut utiliser plusieurs types d'algorithmes pour gérer les processus.

L'algorithme d'ordonnancement par "ordre de soumission" est un algorithme de type FIFO (First In First Out), il utilise donc une file.

22. Nommer une structure de données linéaire de type LIFO (Last In First Out)

À chaque processus, on associe un instant d'arrivée (instant où le processus demande l'accès au processeur pour la première fois) et une durée d'exécution (durée d'accès au processeur nécessaire pour que le processus s'exécute entièrement).

Par exemple, l'exécution d'un processus P4 qui a un instant d'arrivée égal à 7 et une durée d'exécution égale à 2 peut être représentée par le schéma suivant :

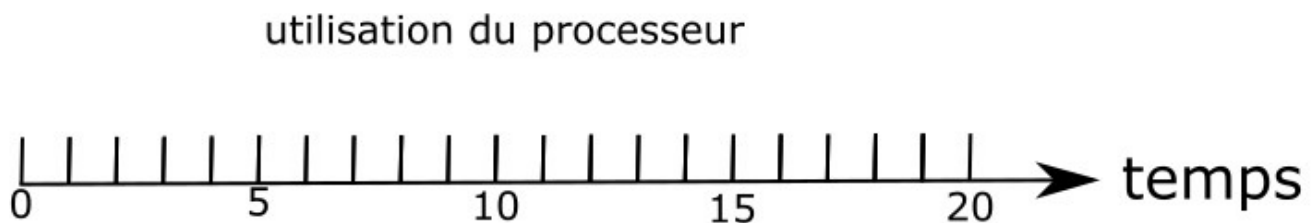


L'ordonnanceur place les processus qui ont besoin d'un accès au processeur dans une file, en respectant leur ordre d'arrivée (le premier arrivé étant placé en tête de file). Dès qu'un processus a terminé son exécution, l'ordonnanceur donne l'accès au processus suivant dans la file.

Le tableau suivant présente les instants d'arrivées et les durées d'exécution de cinq processus :

5 processus		
Processus	instant d'arrivée	durée d'exécution
P1	0	3
P2	1	6
P3	4	4
P4	6	2
P5	7	1

23. Compléter le schéma ci-dessous avec les processus P1 à P5 en utilisant les informations présentes dans le tableau ci-dessus et l'algorithme d'ordonnancement "par ordre de soumission".

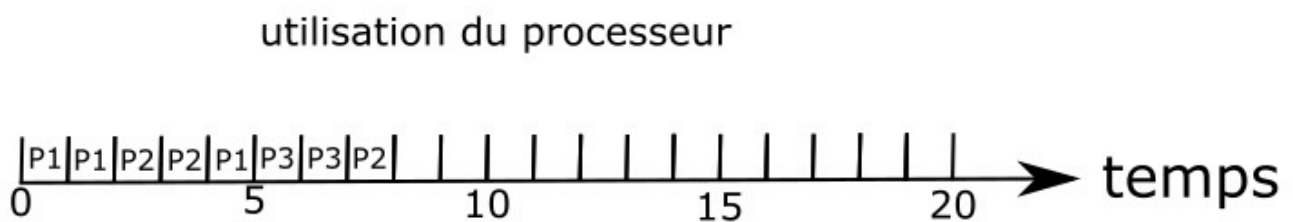


PARTIE C

On utilise maintenant un autre algorithme d'ordonnancement : l'algorithme d'ordonnancement "par tourniquet". Dans cet algorithme, la durée d'exécution d'un processus ne peut pas dépasser une durée Q appelée quantum et fixée à l'avance. Si ce processus a besoin de plus de temps pour terminer son exécution, il doit retourner dans la file et attendre son tour pour poursuivre son exécution.

Par exemple, si un processus P1 a une durée d'exécution de 3 et que la valeur de Q a été fixée à 2, P1 s'exécutera pendant deux unités de temps avant de retourner à la fin de la file pour attendre son tour ; une fois à nouveau élu, il pourra terminer de s'exécuter pendant sa troisième et dernière unité de temps d'exécution.

24. Compléter alors le schéma ci-dessous dans le cas de l'ordonnancement par tourniquet

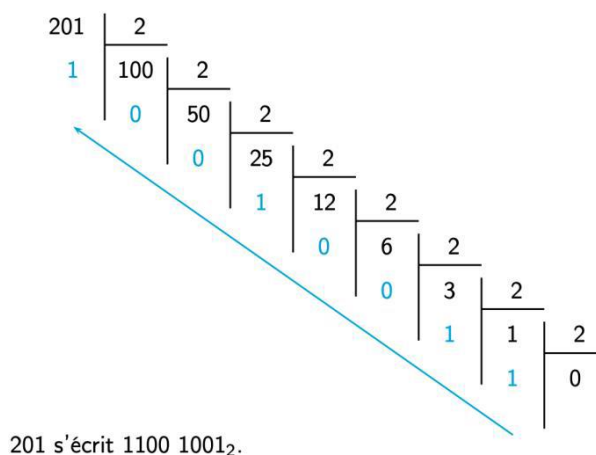


25. L'objectif de cet exercice est de créer une fonction `binaire(a)` qui prend en paramètre un entier positif a en écriture décimale. Cette fonction renvoie l'écriture binaire de a sous forme d'une liste de valeurs 0 ou 1.

L'algorithme utilise la méthode des divisions euclidiennes successives comme l'illustre l'exemple suivant :

Ainsi `binaire(201)` devra renvoyer la liste

`[1, 1, 0, 0, 1, 0, 0, 1]`



On rappelle que

- $a \% 2$ donne le reste de la division euclidienne de a par 2 et
- $a // 2$ donne le quotient de la division euclidienne de a par 2

- a. Écrire une version itérative `binaire_iteratif(a)` qui permet de résoudre ce problème

```
def binaire_iteratif(a):
```

- b. On souhaite maintenant créer une fonction récursive qui permet de résoudre ce problème. Compléter le script suivant

```
def binaire(a):
    return binaire_rec(a, [])

def binaire_rec(a, liste):
    if ...
        return liste
    else:
        r = ...
        a = ...
        .....
        return binaire_rec(.....)
```

26. On considère un jeu de scrabble où le but est de former un mot avec des lettres que l'on a piochées au hasard. Chaque lettre aura une valeur correspondant à sa rareté dans la langue française. Ainsi on pourra fabriquer un dictionnaire dont les clés sont les lettres sous forme de chaîne de caractères et les valeurs une liste contenant le nombre de points attribués à la lettre et le nombre de lettres de ce type. On donne ici le dictionnaire que nous pourrions utiliser :

```
dico = {'A':[1,9], 'B':[3,2], 'C':[3,2], 'D':[2,3], 'E':[1,15], 'F':[4,2],  
'G':[2,2], 'H':[4,2], 'I':[1,8], 'J':[8,1], 'K':[10,1], 'L':[1,5], 'M':[2,3],  
'N':[1,6], 'O':[1,6], 'P':[3,2], 'Q':[8,1], 'R':[1,6], 'S':[1,6], 'T':[1,6],  
'U':[1,6], 'V':[4,2], 'W':[10,1], 'X':[10,1], 'Y':[10,1], 'Z':[10,1]}
```

Ainsi `dico['A']` donnera `[1,9]` car la lettre A vaut 1 point et qu'il y en a 9 dans la pioche

- a. Compléter la fonction `choix_lettre(ALPHABET)` qui choisit une lettre au hasard parmi les lettres de l'alphabet latin transmis sous forme de chaîne de caractère.
On rappelle que la fonction `randint(a,b)` permet de choisir aléatoirement un nombre entre a et b inclus.

```
from random import *  
  
ALPHABET = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'  
  
def choix_lettre():  
    choix = randint(  
    return
```

- b. Compléter la fonction `tirage_lettres(n)` qui simulera le tirage aléatoire de n lettres. Cette fonction fera appel à une fonction récursive `tirage_lettres_rec(n, L, dico)` où L sera la liste contenant les différentes lettres piochées. Ainsi `tirage_lettres(5)` pourra renvoyer :
- ```
[('H', 4), ('M', 2), ('Y', 10), ('W', 10), ('B', 3)]
```

La liste sera donc composée de tuples dont la première valeur sera le nom de la lettre (type str) et la deuxième valeur le nombre de points attribuée à cette lettre (type int)

On initialisera la liste L à `[]` et on insèrera les tuples au fur et à mesure à cette liste.

```
def tirage_lettres(n):
 if n == 0:
 return None
 else:
 return tirage_lettres_rec(n, [], dico)
```

```
def tirage_lettres_rec(n, L, dico):
```

### **Exercice 6 - Facultatif**

27. On considère que nous avons à notre disposition une liste de mots présents dans le dictionnaire français. Par exemple cette liste pourrait contenir entre autres :

```
mots_francais = ['ELEVE', ... , 'INFORMATIQUE' ... , 'NUMERIQUE', ...]
```

Ces mots sont classés par ordre alphabétique.

On souhaite créer une fonction `trouve_mot(lettres, mots_francais)` qui prendra en paramètre une chaîne de caractères de type `str` en majuscule et le « dictionnaire » français et renverra un tuple contenant tous les mots que l'on peut fabriquer avec les lettres de la variable `lettres`.

Ainsi `trouve_mot('ACRSTU')` renverra un tuple dans lequel il y aura entre autres les mots suivants :

'SCRUTA', 'SUCRA', 'CURAT', 'TRACS', 'TRUCS', 'TURCS', 'CAS', 'RAT', 'SU', 'AU', 'AS', Etc.

Vous avez toute liberté pour fabriquer votre programme, toute trace de recherche sera valorisée.



